Projeto TaskTracker

Fase 1 — Planejamento Lógico e Arquitetural · Bootcamp II

Campo	Informação
Estudante	Arthur Grangeiro Botelho
Curso	Análise e Desenvolvimento de Sistemas — UniCEUB
Polo	[preencher: Asa Norte ou Taguatinga]
Projeto	TaskTracker — gerenciador de tarefas via linha de comando

1. Definição do problema

Pessoas que precisam organizar atividades do dia a dia frequentemente perdem prazos por não terem um registro centralizado e padronizado de suas tarefas. O TaskTracker resolve esse problema oferecendo uma aplicação simples, executada no terminal, que permite cadastrar tarefas com informações essenciais (título, descrição, prioridade, prazo e status) e consultá-las de forma organizada.

Escopo desta versão: cadastro e listagem de tarefas em memória durante a execução, com validação de entradas. Persistência em arquivo, edição, exclusão e conclusão de tarefas ficam fora do escopo das Fases 1 e 2.

2. Decomposição do problema

O problema foi dividido em subproblemas menores e independentes, cada um correspondendo a uma função no código:

Subproblema	Responsabilidade	Função planejada
Navegação	Exibir o menu e repetir até o usuário escolher sair	exibir_menu(), main()
Entrada do título	Ler o título e rejeitar valores vazios ou só com espaços	ler_titulo()
Entrada da prioridade	Aceitar somente Alta, Média/Media ou Baixa	ler_prioridade()
Cadastro	Montar o registro da tarefa e adicioná-lo à lista	cadastrar_tarefa()
Listagem	Exibir todas as tarefas ou avisar que a lista está vazia	listar_tarefas()
Encerramento	Finalizar a aplicação com mensagem de saída	opção 3 em main()

3. Arquitetura de dados

Cada tarefa é representada como um **dicionário** (estrutura chave → valor). Todas as tarefas ficam armazenadas em uma **lista** mantida em memória enquanto o programa está em execução.

Campo (chave)	Tipo	Obrigatório	Regra de validação
titulo	String	Sim	Não pode ser vazio nem composto apenas por espaços; repetir a pergunta até ser válido.
descricao	String	Não	Texto livre.
prioridade	String	Sim	Aceita somente “Alta”, “Média” (ou “Media”) e “Baixa”; outra entrada gera erro e repete a pergunta.
data_limite	String	Não	Formato livre, sugerido DD/MM/AAAA.
status	String	Automático	Recebe “Pendente” no momento do cadastro.

Exemplo de registro:

```
tarefa = {
    "titulo":      "Estudar Git",
    "descricao":   "Revisar comandos de commit e push",
    "prioridade":  "Alta",
    "data_limite": "30/11/2026",
    "status":      "Pendente"
}
tarefas = [tarefa, ...]   # lista com todas as tarefas
```

4. Algoritmo em português

Receita lógica que será traduzida para Python na Fase 2:

```
ALGORITMO TaskTracker

INÍCIO
    tarefas ← lista vazia

    REPITA
        MOSTRE "1. Cadastrar nova tarefa"
        MOSTRE "2. Visualizar tarefas cadastradas"
        MOSTRE "3. Sair da aplicação"
        LEIA opcao

        SE opcao = "1" ENTÃO
            REPITA
                LEIA titulo
                SE titulo (sem espaços nas pontas) estiver vazio ENTÃO
                    MOSTRE "Erro: título obrigatório"
            ATÉ QUE titulo não esteja vazio

            LEIA descricao

            REPITA
                LEIA prioridade
                SE prioridade NÃO ESTÁ EM {Alta, Média, Media, Baixa} ENTÃO
                    MOSTRE "Erro: prioridade inválida"
            ATÉ QUE prioridade seja válida

            LEIA data_limite
            status ← "Pendente"
            tarefa ← {titulo, descricao, prioridade, data_limite, status}
            ADICIONE tarefa EM tarefas
            MOSTRE "Tarefa cadastrada com sucesso"
```

```
        SENÃO SE opcao = "2" ENTÃO
            SE tarefas estiver vazia ENTÃO
                MOSTRE "Não existem tarefas cadastradas no momento"
            SENÃO
                PARA CADA tarefa EM tarefas FAÇA
                    MOSTRE título, descrição, prioridade,
                           data limite e status da tarefa
                FIM PARA
            FIM SE

        SENÃO SE opcao = "3" ENTÃO
            MOSTRE "Encerrando o TaskTracker"

        SENÃO
            MOSTRE "Opção inválida"
        FIM SE
    ATÉ QUE opcao = "3"
FIM
```

5. Arquitetura do repositório

```
tasktracker-projeto/
├── README.md                  # Documentação e instruções de uso
├── .gitignore                 # Ignora __pycache__/, *.pyc e .venv/
├── docs/
│   └── planejamento_logico.pdf  # Este documento (Fase 1)
└── src/
    └── main.py                # Código-fonte da aplicação em Python
```

6. Mapeamento para Git/GitHub

O histórico de versões foi planejado para refletir a evolução incremental do projeto:

Etapa	Comando	O que é registrado
1. Inicialização	git init	Criação do repositório local e da estrutura de pastas.
2. Commit inicial	git add / git commit	README.md, .gitignore e docs/planejamento_logico.pdf.
3. Desenvolvimento	git commit	src/main.py com o menu e o loop principal.
4. Regras e validações	git commit	Funções de validação do título e da prioridade.
5. Publicação remota	git remote add origin / git push	Envio do histórico para o GitHub na branch main.

7. Tecnologias

Python 3 (biblioteca padrão), Git, GitHub e terminal (CMD/PowerShell). Na Fase 3 a aplicação será empacotada e containerizada com Docker.